

四電一丙 數位邏輯實習

第 5 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻: 解題、digital、Quartus、協助隊友排錯、拍照(作業一)、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻: 解題、digital、quartus、拍照(作業二)、作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

利用 VHDL Component 重新實作 4 位元的加減法器，將兩個四位元的輸入 X 與 Y 進行加法或減法，並輸出比較結果，請先用 VHDL 條件式完成 1 位元加減法器，再透過 VHDL Component 接成 4 位元加減法器。(本題限定輸入真值表完成 VHDL，不可使用表達式)

B. 作業 2

利用投影片 32-38 頁的資料型態重新設計作業一的加減法器，輸入腳位：四位元的 A 、四位元的 B 、一位元的 Sub ，輸出腳位：四位元的 S ，本題先用 VHDL 實作電路，電路必須使用 VHDL 撰寫，並於報告中解釋程式的每個區塊。

II. 設計程序

A. 作業 1

有別於 Lab02 的加減法器使用 Cin 作為控制線與其中一訊號做 XOR 執行 2 補數計算，本題控制加減法的輸入另外使用了訊號線 $Subtract$ ， Cin 則在 $Sub=0$ 時作為進位輸入； $Sub=1$ 時作為借位輸入。

1. 依據投影片提供的範例，使用 Digital 建立 1bit Adder-Subtractor 的真值表(輸入： A 、 B 、 Cin 、 Sub ；輸出： Sum 、 $Cout$)。(當 $Sub=0$ ， $\{Cout, Sum\} = A + B + Cin$ ； $Sub=1$ ， $\{Cout, Sum\} = A - B - Cin$)

表 1. 1bit Adder-Subtractor 真值表

A	B	Cin	Sub	$Cout$	Sum
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	1	1	1
0	1	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	0
1	0	1	1	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	1	0	1	1
1	1	1	1	1	1

2. 開啟新的分頁，運用 External 元件直接編寫 VHDL，並使用 GHDL 進行編譯。(不需合成 SOP 推得代數式，可直接用真值表，以類似 Multiplexer 的方式完成單一 Adder-Subtractor，再直接以 VHDL 將寫好的單顆 Adder-Subtractor 串接，以完成 4-bit Adder-Subtractor。)

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

-- Top Module : 4位元加減法器
entity FullAdder4 is
port (
    A, B: in std_logic_vector(3 downto 0); -- 4位元運算元
    Ci: in std_logic; -- 初始進位/借位輸入
    Sub: in std_logic; -- 控制訊號(0為加法;1為減法)
    S: out std_logic_vector(3 downto 0); -- 4位元運算結果
    Co: out std_logic; -- 最終進位/借位輸出
end FullAdder4;

-- 4位元加減法器的結構
architecture Structure of FullAdder4 is
-- 宣告1位元加減法器以便呼叫
component FullAdder
port ( A, B, Cin, Sub: in std_logic;
      Cout, Sum: out std_logic );
end component;

-- 宣告內部訊號C，用來作為各個1位元加減法器之間的進位/借位傳遞線(C1, C2, C3)
signal C: std_logic_vector(3 downto 1);
begin
-- 實體化四個 1 位元加減法器，並使用 port map 進行接線 (對應圖一的方塊圖)
-- 共用Sub訊號
FA0: FullAdder port map(A(0), B(0), Ci, Sub, C(1), S(0)); -- 第 0 位元 (LSB 最低位)
FA1: FullAdder port map(A(1), B(1), C(1), Sub, C(2), S(1)); -- 第 1 位元
FA2: FullAdder port map(A(2), B(2), C(2), Sub, C(3), S(2)); -- 第 2 位元
FA3: FullAdder port map(A(3), B(3), C(3), Sub, Co, S(3)); -- 第 3 位元 (MSB 最高位)
end Structure;
```

圖 1. 4-bit Adder-Subtractor VHDL

```
-- Bottom Component : 1位元加減法器
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

-- 宣告1位元加減法器
entity FullAdder is
port (
    A, B, Cin, Sub: in std_logic; -- 1 位元輸入與控制訊號
    Sum, Cout: out std_logic; -- 1 位元輸出
end FullAdder;

architecture WithSelect of FullAdder is

-- 宣告4位元的內部訊號sel，用來將四個輸入打包，方便用於查表索引
signal sel : std_logic_vector(3 downto 0);

begin

-- 將四個輸入串接成選擇向量
sel <= A & B & Cin & Sub;

-- 列出所有Sum會輸出'1'的條件，其餘情況輸出 '0'
with sel select
    Sum <= '1' when "0010", -- A=0 B=0 Cin=1 Sub=0 (加法: 0+0+1 = 和1)
           '1' when "0011", -- A=0 B=0 Cin=1 Sub=1 (減法: 0-0-1 = 差1, 需借位)
           '1' when "0100", -- A=0 B=1 Cin=0 Sub=0 (加法: 0+1+0 = 和1)
           '1' when "0101", -- A=0 B=1 Cin=0 Sub=1 (減法: 0-1-0 = 差1, 需借位)
           '1' when "1000", -- A=1 B=0 Cin=0 Sub=0 (加法: 1+0+0 = 和1)
           '1' when "1001", -- A=1 B=0 Cin=0 Sub=1 (減法: 1-0-0 = 差1)
           '1' when "1110", -- A=1 B=1 Cin=1 Sub=0 (加法: 1+1+1 = 和1, 有進位)
           '1' when "1111", -- A=1 B=1 Cin=1 Sub=1 (減法: 1-1-1 = 差1, 需借位)
           '0' when others; -- 涵蓋剩下的 8 種組合

-- 列出所有Cout會輸出'1'的條件
with sel select
    Cout <= '1' when "0011", -- A=0 B=0 Cin=1 Sub=1 (減法: 0 不夠減 1, 產生借位)
              '1' when "0101", -- A=0 B=1 Cin=0 Sub=1 (減法: 0 不夠減 1, 產生借位)
              '1' when "0110", -- A=0 B=1 Cin=1 Sub=0 (加法: 0+1+1 = 2, 產生進位)
              '1' when "0111", -- A=0 B=1 Cin=1 Sub=1 (減法: 0 不夠減 (1+1), 產生借位)
              '1' when "1010", -- A=1 B=0 Cin=1 Sub=0 (加法: 1+0+1 = 2, 產生進位)
              '1' when "1100", -- A=1 B=1 Cin=0 Sub=0 (加法: 1+1+0 = 2, 產生進位)
              '1' when "1110", -- A=1 B=1 Cin=1 Sub=0 (加法: 1+1+1 = 3, 產生進位)
              '1' when "1111", -- A=1 B=1 Cin=1 Sub=1 (減法: 1 不夠減 (1+1), 產生借位)
              '0' when others; -- 涵蓋剩下的 8 種組合
end WithSelect;
```

圖 2. 1-bit Adder-Subtractor VHDL

VHDL 程式碼講解：

紅色框：是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

橘色框：分別定義 4-bit Adder-Subtractor、1-bit Adder-Subtractor 的外部接腳：

4-bit Adder-Subtractor：包含兩個 4 位元陣列輸入 (A, B)、初始進/借位 (Ci)、控制訊號 (Sub)，以及 4 位元陣列結果 (S) 與最終進/借位輸出 (Co)。

1-bit Adder-Subtractor：包含四個單一輸入 (A, B, Cin, Sub) 與兩個單一輸出 (Sum, Cout)。此介面會被 4 位元頂層模組呼叫。

黃色框：程式的核心運算區，定義輸入與輸出間關係：

4-bit Adder-Subtractor：利用 component 宣告 1 位元加法器元件，並宣告內部訊號 C 作為進位傳遞線。接著利用 port map 將四個 1 位元元件串接起來（漣波進位，**模擬實際電路圖中 IC 串接的硬體架構。(結構化描述 (Structural))**

1-bit Adder-Subtractor：將輸入打包成 sel 向量，利用 with...select 語法實作真值表。**程式各區塊為平行處理沒有先後順序，模擬硬體電路中輸入訊號一改變、輸出立即反應的特性。(行為描述 (Behavioral/Dataflow))**

注意：

另外，VHDL 中 LIBRARY 宣告的有效範圍只針下面出現一組 Entity + Architecture 有效，因此才會重複呼叫，

並且不可以更動 1-bit Adder-Subtractor 與 4-bit Adder-Subtractor 上下關係，否則執行結果不同。

3. 完成 External，檢查概念(可以運用真值表推得的代數式檢查)，測試不同輸入組合，確認輸出結果符合原先表示式後匯出 VHDL 檔案。

$$\begin{aligned} Sum &= \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C_{in}} + A\overline{B}\overline{C_{in}} + ABC_{in} \\ &= C_{in}(\overline{A}\overline{B} + AB) + \overline{C_{in}}(\overline{A}B + A\overline{B}) \\ &= C_{in}(\overline{A \oplus B}) + \overline{C_{in}}(A \oplus B) \\ &= C_{in}\overline{X} + \overline{C_{in}}X \quad (X = A \oplus B) \\ &= X \oplus C_{in} \\ &= A \oplus B \oplus C_{in} \end{aligned}$$

圖 3. 1-bit Adder-Subtractor Sum 布林代數式推導

$$C_{out} = \overline{A}C_{in}Sub + \overline{A}B\overline{Sub} + AC_{in}\overline{Sub} + AB\overline{Sub} + BC_{in}$$

圖 4. 1-bit Adder-Subtractor Cout 布林代數式推導

Sum 的式子裡面完全沒有 Sub 是因為在二進位運算中，無論是加法還是減法該位元的和或差邏輯是一樣的，例如：1+0=1, 1-0=1；1+1=0 進位 1, 1-1=0 不借位，所以 Sum 不受 Sub 切換的影響。

Cout 式子可以分為 Sub = 0 / 1 兩種情況：

$$\begin{aligned}
C_{out} &= \overline{A}C_{in}(0) + \overline{A}B(0) + AC_{in}(1) + AB(1) + BC_{in} \\
&= 0 + 0 + AC_{in} + AB + BC_{in} \\
&= AB + BC_{in} + AC_{in}
\end{aligned}$$

圖 5. addition mode Cout 布林代數式推導

$$\begin{aligned}
C_{out} &= \overline{A}C_{in}(1) + \overline{A}B(1) + AC_{in}(0) + AB(0) + BC_{in} \\
&= \overline{A}C_{in} + \overline{A}B + 0 + 0 + BC_{in} \\
&= \overline{A}B + \overline{A}C_{in} + BC_{in}
\end{aligned}$$

圖 6. Subtraction mode Cout 布林代數式推導

符合標準全加器 (Full Adder) 的進位 (Carry-out)、全減法器 (Full Subtractor) 的借位 (Borrow-out) 公式。

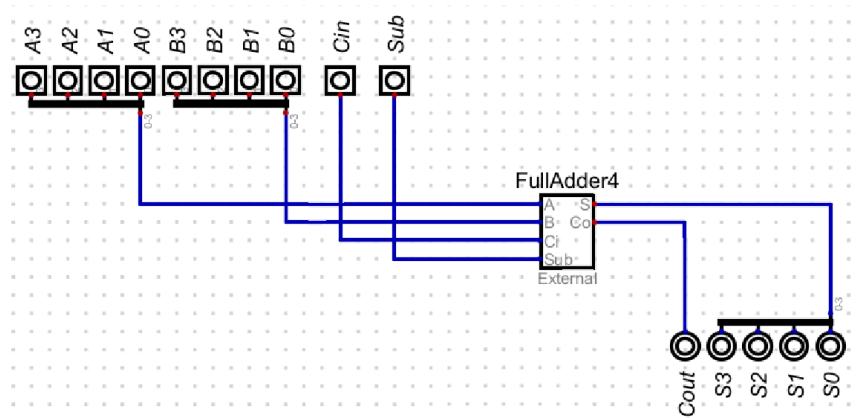


圖 7. 4-bit Adder-Subtractor 模擬電路

B. 作業 2

主要以資料型態轉換與運算來簡化成高階數學運算，大幅簡化程式碼，相較於作業一需要定義多個 Component 並串接以及進行冗長的行為描述，作業二透過型態轉換後只需簡單的加減算式即可達到類似效果(題目拔除 Cout，會少一位進位/借位結果，表達若相減為負需+16)。

1. 運用 External 元件，參考投影片提供的範例編寫 VHDL，並使用 GHDL 進行編譯。

```
library IEEE;
use IEEE.std_logic_1164.all;
-- 必須引入numeric_std函式庫使用後面的to_integer, unsigned, to_unsigned等數值轉換功能
use IEEE.numeric_std.all;

-- =====
-- 定義
-- =====
entity FullAdder4 is
  port(
    A : in std_logic_vector(3 downto 0); -- 4位元輸入
    B : in std_logic_vector(3 downto 0); -- 4位元輸入
    Ci : in std_logic; -- 加減法控制訊號('0'為加法, '1'為減法)
    S : out std_logic_vector(3 downto 0) -- 4位元輸出結果
  );
end FullAdder4;

-- =====
-- 行為描述
-- =====
architecture Structure of FullAdder4 is
  -- 宣告內部訊號使用integer，預設值為0，用來暫存轉換成十進位數字後的A, B 以及計算結果
  signal A_int, B_int, S_int : integer := 0;
begin
  -- 步驟 1: 【輸入資料型態轉換】 (位元向量 -> 整數)
  -- VHDL規定std_logic_vector不能直接做數學加減，所以要先轉型。
  -- 先用unsigned()宣告它們是無號數，再用to_integer()轉成十進位整數放入_int訊號中。
  A_int <= to_integer(unsigned(A));
  B_int <= to_integer(unsigned(B));

  -- 步驟 2: 【數學運算】 (條件式加減法)
  -- 利用when...else語法直接做高階的數學整數運算。
  -- 當Ci為'0'時執行(A_int + B_int)，否則執行(A_int - B_int)，並把結果存入 S_int。
  S_int <= (A_int + B_int) when Ci = '0' else (A_int - B_int + 16);

  -- 步驟 3: 【輸出資料型態轉換】 (整數 -> 位元向量)
  -- S是 std_logic_vector型態，所以要把算出來的整數S_int轉回去。
  -- 先用to_unsigned將S_int轉為4位元無號數，最後轉成std_logic_vector輸出。
  S <= std_logic_vector(to_unsigned(S_int, 4));
end Structure;
```

圖 8. 4-bit Adder-Subtractor VHDL _ Q2

VHDL 程式碼講解：

紅色框：Library 宣告區，載入 IEEE 基礎資料庫。除了定義邏輯閘與標準語法的 std_logic_1164 之外，這裡特別多載入了一行 numeric_std 函式庫，這是為了讓後續程式能夠使用數值型態轉換的功能。

橘色框：是介面定義，定義有 A、B 有四個輸入依序最高位是 3 到最低位 0，Ci 為一輸入 S 輸出是有四個輸出依序最高位是 3 到最低位 0。

黃色框：程式的核心運算區，定義輸入與輸出間關係，如圖先進行資料型態的轉換流程，各行程式間並無順序。S 等於 A 加 B 當 Ci 是 0 時，若不是 S 等於 A 減 B，利用 to_unsigned(S_int, 4) 將運算後的整數轉回長度為 4 的無符號向量，模擬實際電路中邏輯閘關係。

2. 完成 External，測試不同輸入組合，確認輸出結果符合原先表示式後匯出 VHDL 檔案。

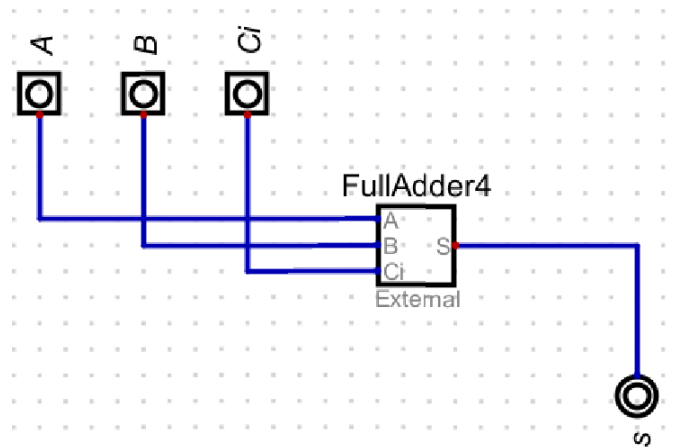


圖 9. 4-bit Adder-Subtractor 模擬電路_Q2

III. 模擬驗證

A. 作業 1

4-bit Adder-Subtractor :

Input 由左至右分別為 A(A3,A2,A1,A0)、B(B3,B2,B1,B0)、Ci、Sub；Output Cout、S(S3,S2,S1,S0)。

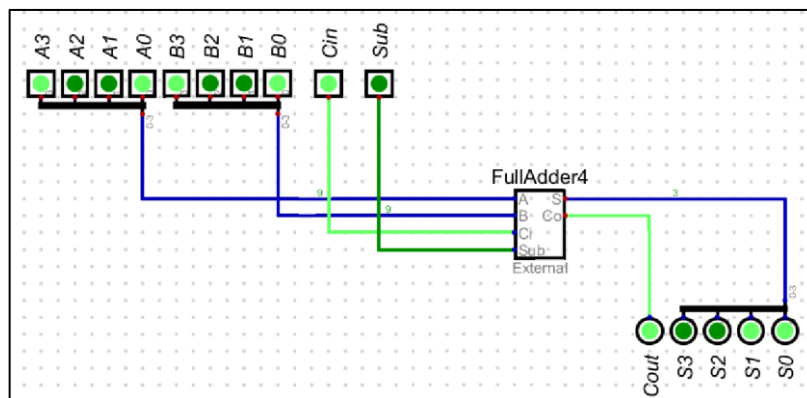


圖 10. Input: 1001+1001+1 / Output: 10011
Decimal : 9+9+1 = 19 (進位)

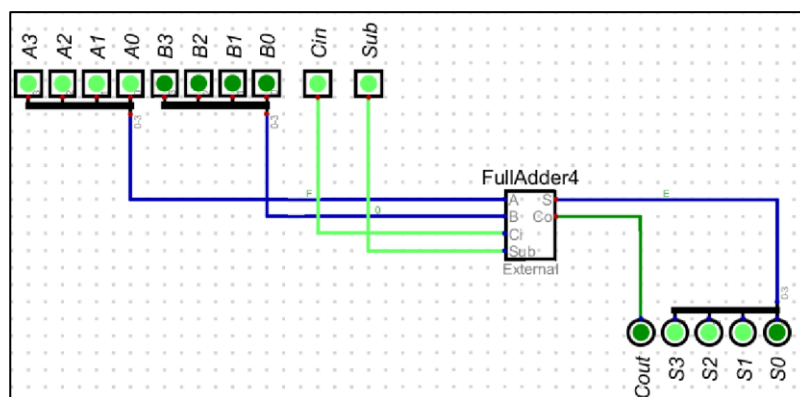


圖 11. Input: 1111-0000-1 / Output: 01110
Decimal : 15-0-1 = 14 (無進位/借位)

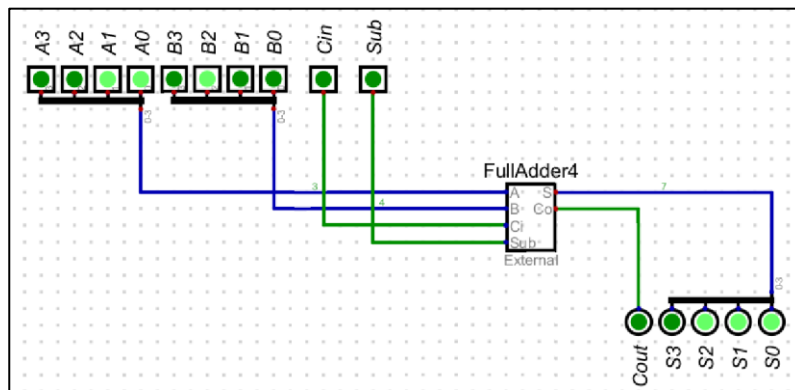


圖 12. Input:0011+0100+0 / Output:0011
 Decimal : 3+4+0 = 7 (無進位/借位)

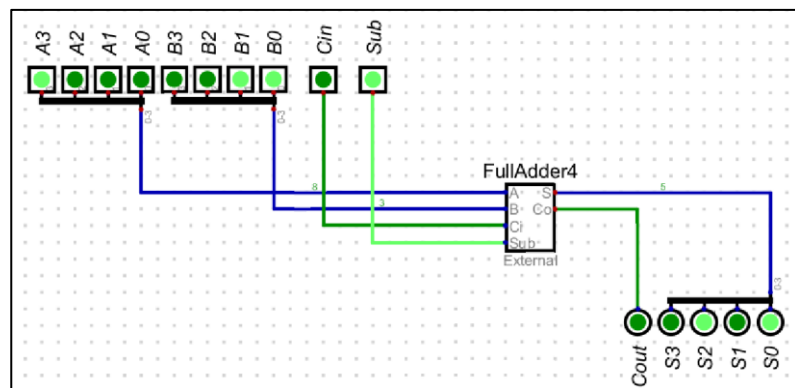


圖 13. Input:1000-0011-0 / Output:0010
 Decimal : 8-3-0 = 5 (無進位/借位)

B. 作業 2

4-bit Adder-Subtractor :

Input 由左至右分別為 A(A3,A2,A1,A0)、B(B3,B2,B1,B0)、Ci(Add/Subtract control signal) ; Output S(S3,S2,S1,S0)。

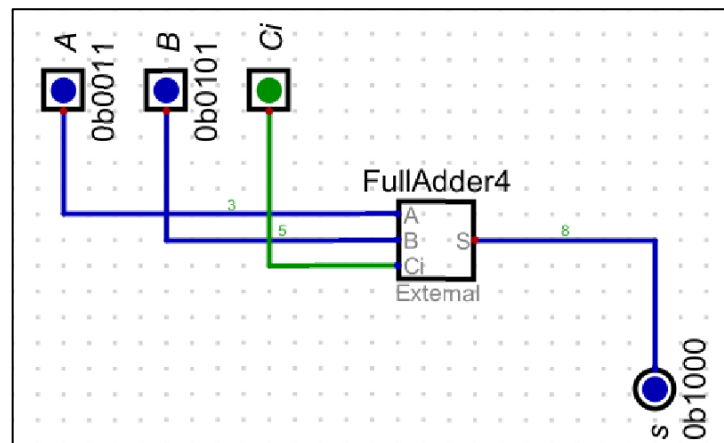


圖 14. Input:0011+0101 / Output:1000
 Decimal : 3+5 = 8 (無進位/借位)

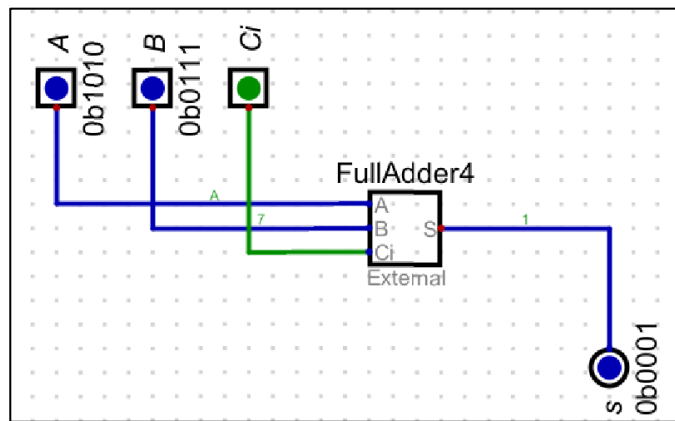


圖 15. Input: 1010+0111 / Output: 0001
Decimal : 10+7= 17 (進位-16=1)

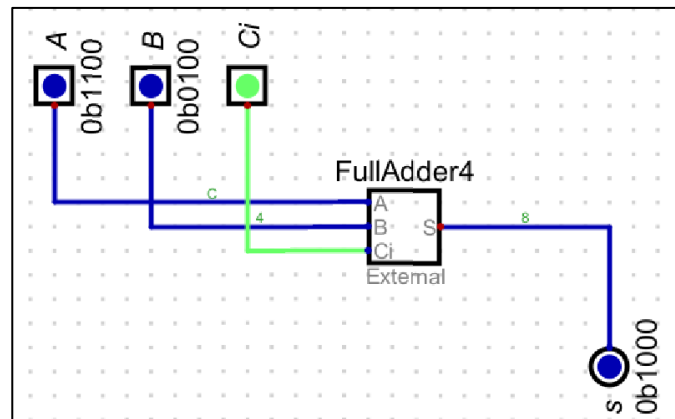


圖 16. Input: 1100-0100 / Output: 1000
Decimal : 12-4 = 8(無進位/借位)

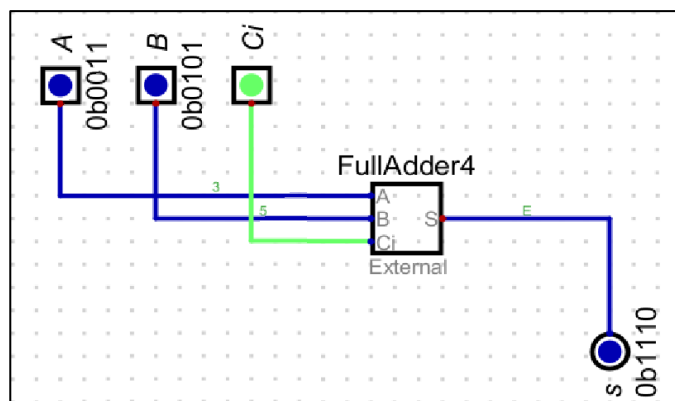


圖 17. Input: 0011+0101 / Output: 1110
Decimal : 3-5 = -2 (借位+16=14)

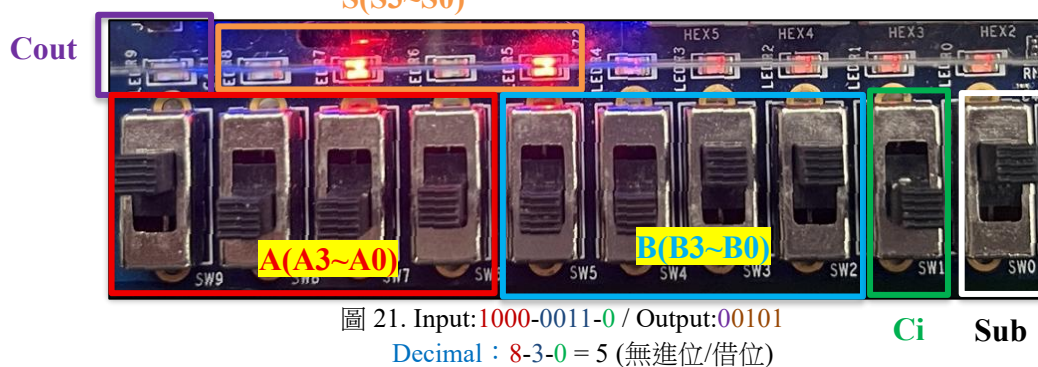
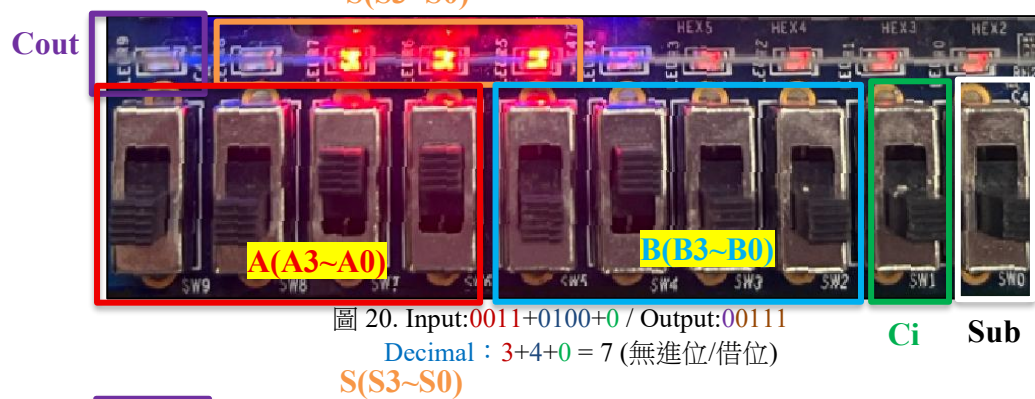
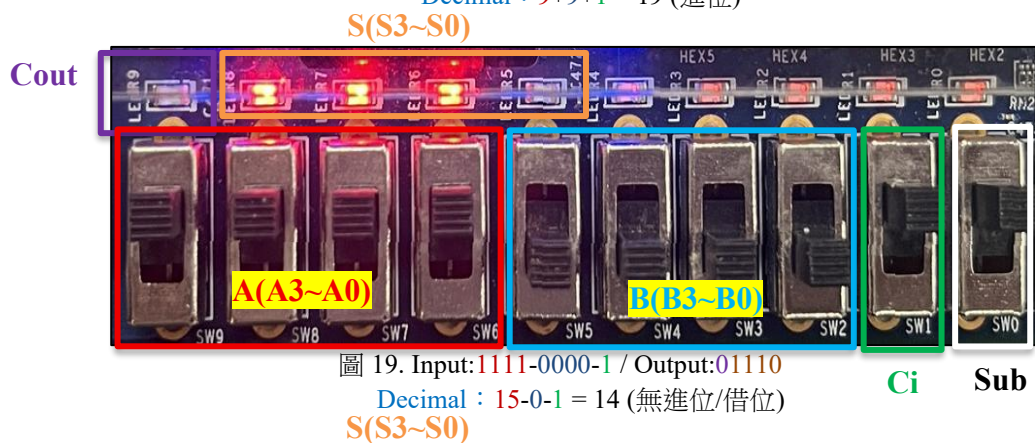
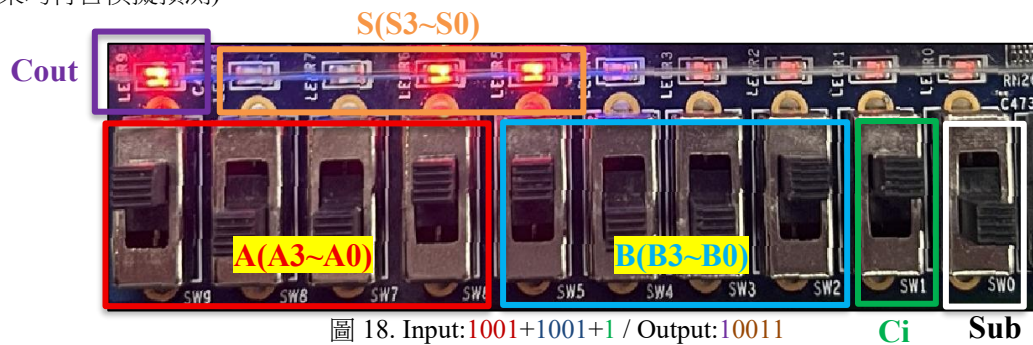
IV. 實驗結果與成果討論

A. 作業 1

4-bit Adder-Subtractor :

Input 由左至右分別為 A(A3,A2,A1,A0)、B(B3,B2,B1,B0)、Ci、Sub ; Output Cout、S(S3,S2,S1,S0)。

輸入 A、B、Ci、Sub，為 sw9~sw0，輸出 Cout、S，為 LED9~LED5。
(以下結果均符合模擬預測)



B. 作業 2

4-bit Adder-Subtractor :

Input 由左至右分別為 A(A3,A2,A1,A0)、B(B3,B2,B1,B0)、Ci(Add/Subtract control signal)；Output S(S3,S2,S1,S0)。

輸入 A、B、Ci(Add/Subtract control signal)，為 sw9~sw1，輸出 S，為 LED9~LED6。
 (以下結果均符合模擬預測)

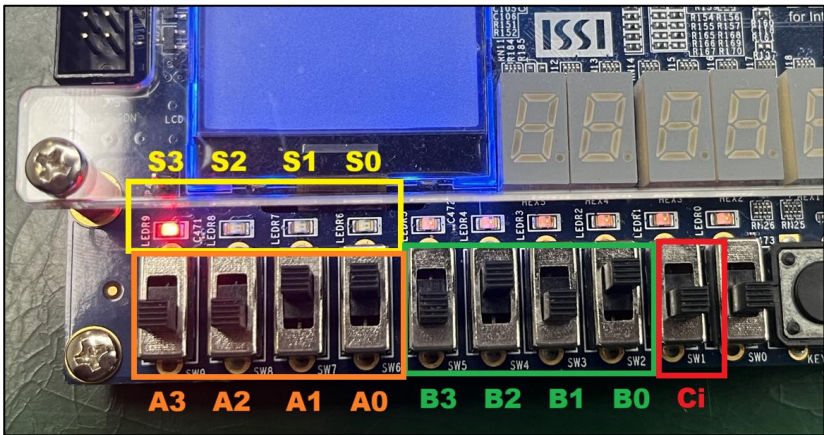


圖 22. Input:0011+0101 / Output:1000
 Decimal : 3+5 = 8 (無進位/借位)

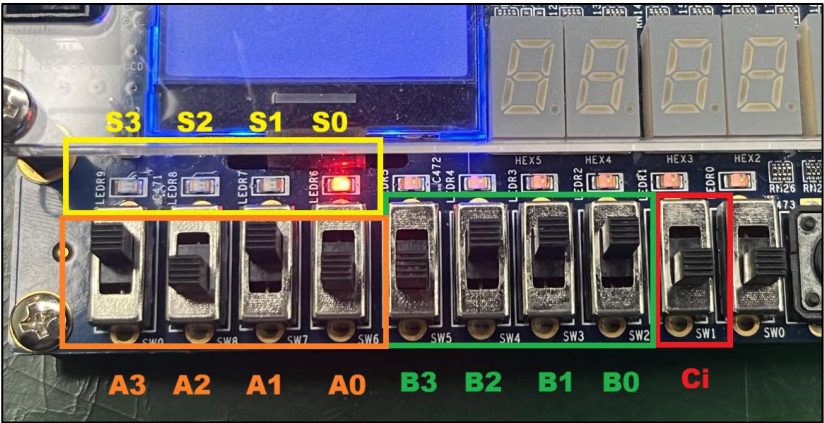


圖 23. Input:1010+0111 / Output:0001
 Decimal : 10+7= 17 (進位-16=1)

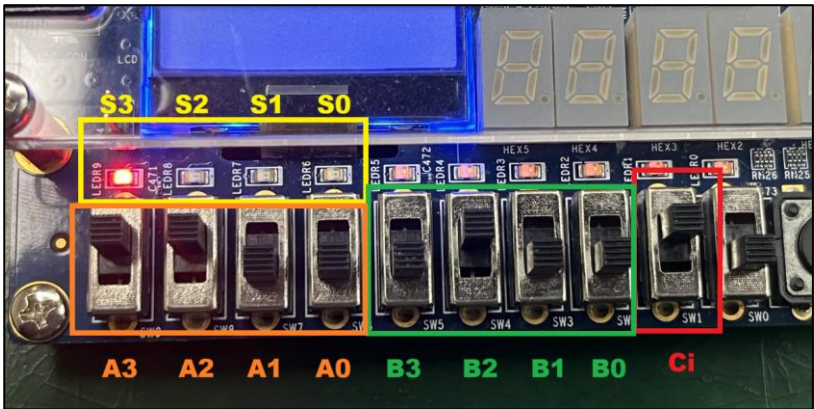


圖 24. Input:1100-0100 / Output:1000
 Decimal : 12-4 = 8(無進位/借位)

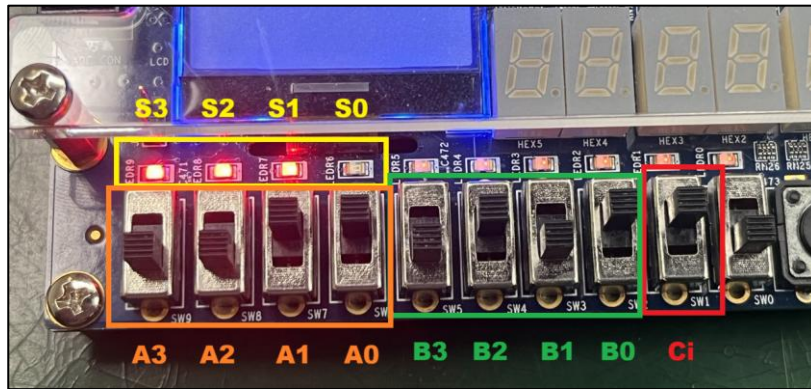


圖 25. Input:0011+0101 / Output:1110
 Decimal : 3-5 = -2 (借位+16=14)

V. 實驗心得

組員 1：搞懂進位、借位關係，認識 VHDL 細節。

組員 2：可以明顯感受到兩題在設計過程中速度差距，第一題因為需要輸入至少一半真值表因此完成時間有受到影響。

參考文獻

- [1] DLE 06 數位邏輯實習 VHDL(II)